Day 87 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 4 reference

Watch: Principal Component Analysis (PCA), Step-by-Step (StatQuest) | t-SNE, Clearly Explained (StatQuest)

`day87_embedding_visualization.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X = the vectors, one row per word · Y = the 2-D positions · p_{ij} = how near j is to i in 50-D · q_{ij} = the same in 2-D

Segment 0 — Setup

Settings and load_vectors

CODE

```
VECTORS = os.path.join(HERE, "day86_vectors.txt.gz")
N_OTHER = 410
PERPLEXITY = 30
TSNE_SEEDS = [0, 1, 2, 3, 4]
K = 10
```

- `PERPLEXITY = 30` — roughly how many neighbours each point's Gaussian covers (t-SNE's usual default).
- `TSNE_SEEDS = [0, 1, 2, 3, 4]` — five random starting layouts: t-SNE's answer depends on where it starts.

Segment 1 — PCA

pca: eigenvectors of the covariance matrix

MATH

$$C = X_c^T X_c / (n-1); \quad C v = \lambda v; \quad \text{share of direction } i = \lambda_i / \Sigma \lambda$$

CODE

```
def pca(X, k):
    Xc = X - X.mean(axis=0)
    C = Xc.T @ Xc / (len(X) - 1)
    vals, vecs = np.linalg.eigh(C)
    order = np.argsort(vals)[::-1]
    vals, vecs = vals[order], vecs[:, order]
    return Xc @ vecs[:, :k], vecs[:, :k], vals / vals.sum()
```

- `np.linalg.eigh(C)` — eigen-decomposition for a symmetric matrix; it returns the eigenvalues smallest first.
- `np.argsort(vals)[::-1]` — the order of largest first; [::-1] reverses an array.
- `Xc @ vecs[:, :k]` — each row's coordinates along the first k directions.

Segment 2 — t-SNE

sq_dists and affinities: t-SNE's P

MATH

$$p_{j|i} = exp(-\beta_i\, d_{ij}) \,/\, \Sigma_k\, exp(-\beta_i\, d_{ik}), \quad \beta_i \text{ chosen so that entropy} = \log(\text{perplexity}); \quad p_{ij} = (p_{j|i} + p_{i|j}) \,/\, 2n$$

CODE

```
def sq_dists(X):
    s = (X ** 2).sum(axis=1)
    return np.maximum(s[:, None] + s[None, :] - 2 * X @ X.T, 0)
def affinities(X, perplexity):
    D = sq_dists(X); n = len(X)
    P = np.zeros((n, n)); target = np.log(perplexity)
    for i in range(n):
        lo, hi, beta = 0.0, np.inf, 1.0
        d = np.delete(D[i], i)
        for _ in range(60):
            p = np.exp(-(d - d.min()) * beta); p /= p.sum()
            H = -(p * np.log(p + 1e-12)).sum()
            if abs(H - target) < 1e-6: break
            if H > target: lo, beta = beta, beta * 2 if hi == np.inf else (beta + hi) / 2
            else: hi, beta = beta, (beta + lo) / 2
        P[i, np.arange(n) != i] = p
    return (P + P.T) / (2 * n)
```

- `s[:, None] + s[None, :] - 2 * X @ X.T` — all squared distances at once, from $|a - b|^2 = |a|^2 + |b|^2 - 2\, a{\cdot}b$.
- `np.delete(D[i], i)` — point i's distances to every other point (itself removed).
- `lo, beta = beta, beta * 2 if hi == np.inf else (beta + hi) / 2` — a tuple assignment: the right side is computed first, then both names change.
- `(P + P.T) / (2 * n)` — symmetric, and summing to 1 over all pairs.

kl_and_grad: the loss and its gradient

MATH

$$q_{ij} = (1 + |y_i - y_j|^2)^{-1} \,/\, \Sigma\,(\ldots); \quad KL = \Sigma\, p\, \log(p \,/\, q)$$

$$dKL/dy_i = 4\, \Sigma_j\, (p_{ij} - q_{ij})(1 + |y_i - y_j|^2)^{-1}\, (y_i - y_j)$$

CODE

```
def kl_and_grad(P, Y):
    num = 1 / (1 + sq_dists(Y)); np.fill_diagonal(num, 0)
    Q = np.maximum(num / num.sum(), 1e-12)
    kl = np.sum(P * np.log(np.maximum(P, 1e-12) / Q))
    W = (P - Q) * num
    grad = 4 * (W.sum(axis=1)[:, None] * Y - W @ Y)
    return kl, grad
```

- `num = 1 / (1 + sq_dists(Y))` — Student-t similarities: heavy tails, so moderately far points can sit far apart in 2-D.
- `W.sum(axis=1)[:, None] * Y - W @ Y` — the sum over j of $W_{ij}(y_i - y_j)$ for every i, as two matrix operations.

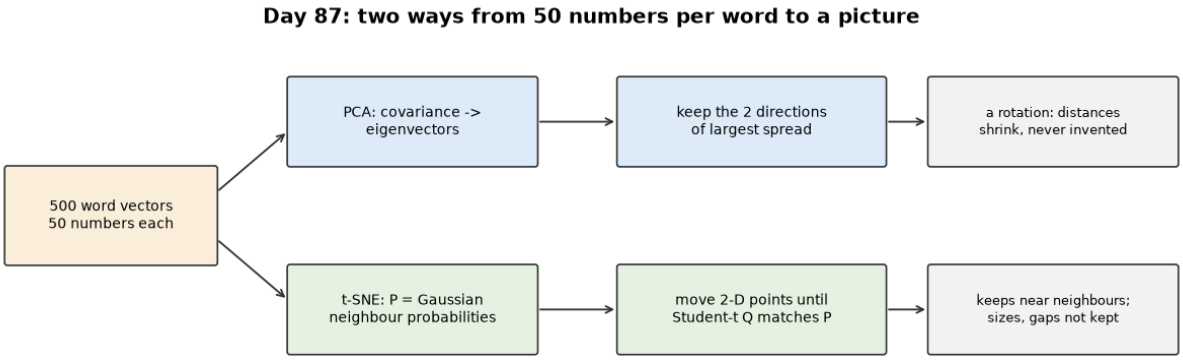
tsne: gradient descent with momentum and gains

CODE

```
def tsne(X, perplexity=PERPLEXITY, seed=0, iters=1000, lr=100.0, exag=12.0, exag_iters=250):
    P = affinities(X, perplexity)
    rng = np.random.RandomState(seed)
    Y = rng.randn(len(X), 2) * 1e-4
    vel, gains = np.zeros_like(Y), np.ones_like(Y)
    for it in range(iters):
        _, g = kl_and_grad(P * (exag if it < exag_iters else 1.0), Y)
        gains = np.where(np.sign(g) != np.sign(vel), gains + 0.2, gains * 0.8).clip(0.01)
        mom = 0.5 if it < 250 else 0.8
        vel = mom * vel - lr * gains * g
        Y = Y + vel; Y -= Y.mean(axis=0)
    return Y, kl_and_grad(P, Y)[0]
```

- `P * (exag if it < exag_iters else 1.0)` — early exaggeration: for the first 250 steps P is 12 times larger, so groups pull together first.
- `np.where(np.sign(g) != np.sign(vel), gains + 0.2, gains * 0.8)` — per-coordinate step sizes grow while the direction holds and shrink when it flips.
- `Y -= Y.mean(axis=0)` — keeps the picture centred; only distances matter.

SYSTEM DIAGRAM



PCA keeps the directions of largest spread; t-SNE keeps near neighbours.

Segment 3 — Measures and experiments

group_share and preserved: numbers for every picture

CODE

```
def group_share(Y, groups):
    D = sq_dists(Y); np.fill_diagonal(D, np.inf)
    return np.mean(groups[D.argmin(axis=1)] == groups)
def preserved(X, Y, k=K):
    def knn(Z):
        D = sq_dists(Z); np.fill_diagonal(D, np.inf)
        return np.argsort(D, axis=1)[:, :k]
    a, b = knn(X), knn(Y)
    return np.mean([len(set(a[i]) & set(b[i])) / k for i in range(len(X))])
```

- `groups[D.argmin(axis=1)] == groups` — for every point, is its nearest point in the same group?
- `np.argsort(D, axis=1)[:, :k]` — each row's k smallest distances, as indices.
- `set(a[i]) & set(b[i])` — the neighbours shared before and after projecting.

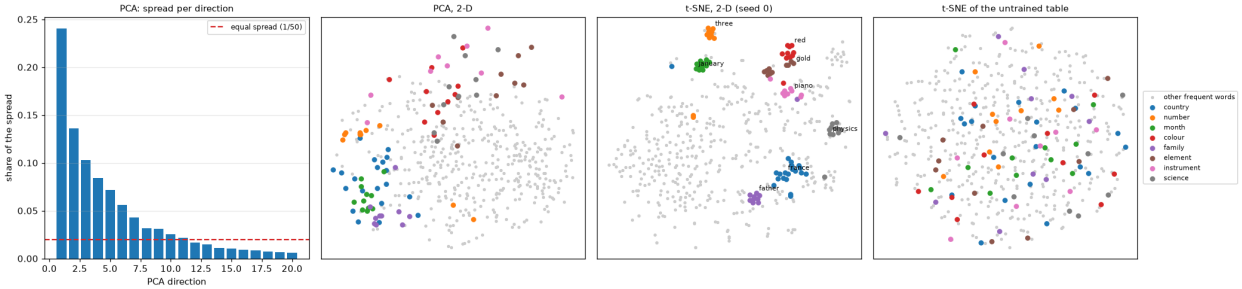
demo_controls (c): clusters with known sizes

CODE

```
centres = np.zeros((3, 50)); centres[1, 0] = 20; centres[2, 0] = -80
spread = [1.0, 0.1, 0.1]
S = np.concatenate([centres[j] + spread[j] * rng.randn(100, 50) for j in range(3)])
lab = np.repeat([0, 1, 2], 100)
out = {}
for name, Z in [("50-D (true)", S), ("PCA, 2-D", pca(S, 2)[0]), ("t-SNE, 2-D", tsne(S,
    seed=0)[0])]:
    c = np.array([Z[lab == j].mean(axis=0) for j in range(3)])
    r = [np.sqrt(((Z[lab == j] - c[j]) ** 2).sum(axis=1)).mean() for j in range(3)]
    out[name] = (r[0] / r[1], np.linalg.norm(c[0] - c[2]) / np.linalg.norm(c[0] - c[1]))
```

- `centres[j] + spread[j] * rng.randn(100, 50)` — 100 Gaussian points around each centre; broadcasting adds the centre to every row.
- `np.concatenate([...])` — the three clusters stacked into one (300, 50) array.
- `r[0] / r[1]` — how much larger cluster A looks than B, in the space being measured.

REAL GRAPH



Real graph from the script: PCA's spread, PCA and t-SNE of 500 words, and t-SNE of the untrained table.

Quick Syntax Glossary — new or reinforced constructs

- `np.linalg.eigh(C)` — eigenvalues and eigenvectors of a symmetric matrix.
- `np.linalg.svd(X, full_matrices=False)` — the singular value decomposition, used to check PCA.
- `a[::-1]` — an array reversed.
- `np.delete(a, i)` — a copy of a without entry i.
- `np.where(cond, a, b)` — a where cond is true, b elsewhere.
- `x, y = y, f(x)` — tuple assignment: the right side is evaluated before either name changes.

End of Day 87 syntax reference. For the experiments and what they showed, see the Day 87 complete guide.